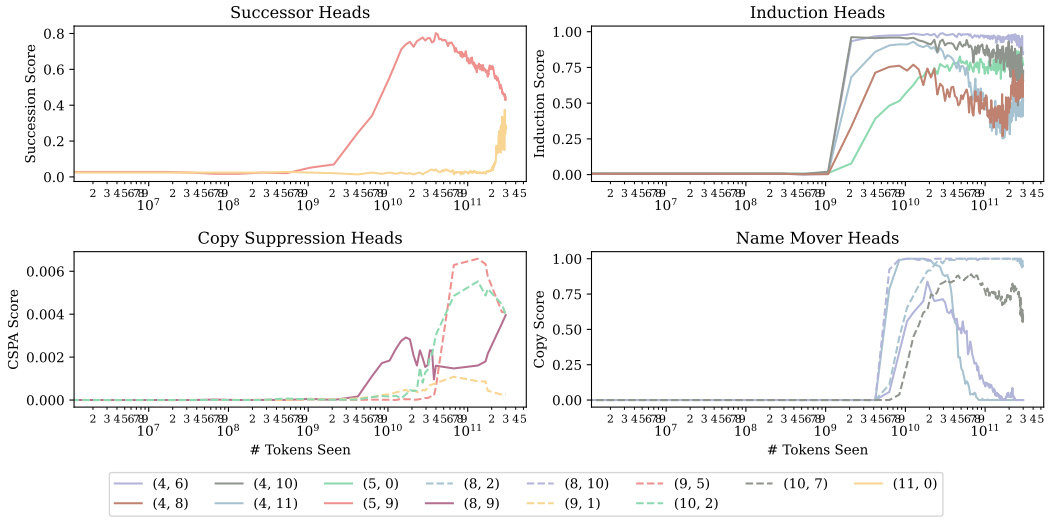


Figure 3: The development over time of components relevant to IOI and Greater-Than in Pythia-160m. Each line indicates the degree to which an attention head, denoted as (layer, head), exhibits a given function; higher values imply stronger functional behavior. Heads often lose their current function; as this occurs, other heads take their place.

Pythia-160m's 300B token training span, we extend beyond previous work, which studies models trained on relatively few ($\leq$ 50M) tokens [14, 63]; in such work, components and task behaviors appear constant after component formation.

By contrast, our results (Figure 3) show that over the longer training period of Pythia models, the identity of components in each circuit is not constant. For example, the name-mover head (4,6) suddenly stops exhibiting this behavior at $3 \times 10^{10}$ tokens, having acquired it after $4 \times 10^{9}$ tokens. Similarly, Pythia-160m's main successor head (5,9) loses its successor behavior towards the end of training; however, (11,0) exhibits more successor behavior at precisely that time. Such balancing may lead to the model's task performance remaining stable, as we observed in the prior section (Figure 1). It seems plausible that self-repair [46, 58] contributes to this behavioral stability, but we leave the question of the exact "load-balancing" mechanism to future work. Nevertheless, models can clearly compensate for losses of and changes in individual circuit components.

### 4.2 Circuit Algorithm Stability Over Training

This instability of functional components raises an important question—when attention heads begin or cease to participate in a circuit, does the underlying algorithm change? To answer this, we examined the IOI circuit, as it is the most thoroughly characterized [71] circuit algorithm of our set of tasks. Our investigation follows a three-stage approach: first, we analyzed the IOI circuit at the end of training, reverse-engineering its algorithm; next, we developed a set of metrics to quantify whether the model was still performing that algorithm; finally, we applied these metrics across checkpoints, to determine if the algorithm was stable over training.

At each checkpoint, we identified relevant heads through two criteria:

1. The head's effect on model performance when ablated via path patching
2. The head's score on component-specific functional tests (e.g., copy suppression score for name-mover heads)

Heads were included in our analysis only if they both: scored above a 10% threshold on their respective functional tests, and showed a negative effect on the logit difference when ablated. For S2-inhibition heads, we additionally verified that ablating positional information while preserving token information reduced both logit difference and name-mover head attention to the indirect object while increasing attention to the subject.

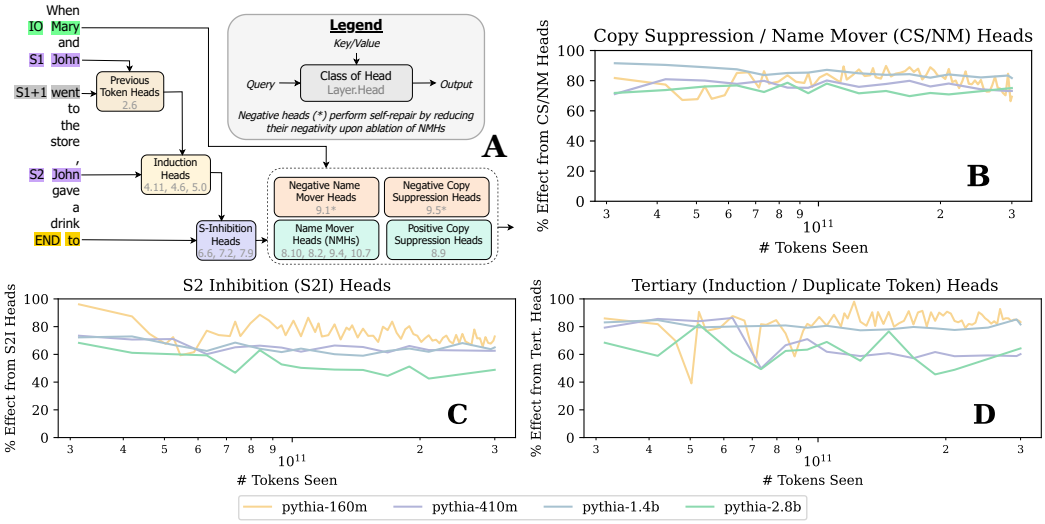


Figure 4: **A**: Pythia-160m's IOI circuit at the end of training (300B tokens). The remaining plots show the percent of model IOI performance that is explained by the Copy Suppression and Name-Mover Heads (**B**), the S-Inhibition Heads' edges to those heads (**C**), and the Induction / Duplicate Token Heads' connections to the S-Inhibition heads (**D**); higher percentages indicate that the corresponding edge is indeed important. Each of plots **B**-**D** verifies the importance of an edge from diagram **A**.

The first stage of our analysis is to analyze the IOI circuit at the end of training. Here, we present only the results of our analysis, but see Appendix B for details of this process, which follows the original analysis [71]. Figure 4**A** shows the circuit that results from our analysis; it involves three logical "steps," each of which involves a different set of attention head types. Working backwards from the logit predictions, the direct contributors towards the logit difference are name-mover heads and copy suppression heads. The former attend to the indirect object in the prompt and copy it to the last position; the latter attend to and downweight tokens that appear earlier in the input. In the next step, the name-mover heads (but not the copy-suppression heads) use on token and positional information output by the S-inhibition heads to attend to the correct token. Finally, S-inhibition heads rely on information from induction heads and duplicate-token heads.

Next, we quantify the extent to which the circuit depends on each of these three steps, via path patching [28]. When using path patching, we intervene on a node H (e.g., S-inhibition heads), route the change through an intermediate node or set of nodes R that depends on H (e.g., name-mover heads), and finally measure how the change in R affects the model output. For each step, our metric is the effect (on logit difference) of ablating the targeted components (through any intermediate nodes) divided by the effect of ablating all components that could affect R. For example, when measuring S-inhibition heads' importance to name-mover heads, the denominator is the ablation effect of all heads upstream of the name-mover heads. Higher ratios (0-100%) indicate greater relative importance of the targeted components. More details on path patching can be found in Appendix B, while details on the algorithm experiment itself can be found in Appendix A.

Finally, we compute each of these metrics for each model from 160M to 2.8B parameters in size.[4] We run them on each checkpoint post-circuit emergence (that is, when all component types appear in the circuit); for Pythia-160m, we test every checkpoint, and for the larger models we space out checkpoints to save compute, using approximately 1/3rd of the available checkpoints). We find (Figure 4**B**-**D**) that the behavior measured by these metrics is stable once the initial circuit has formed. Notably, in no model or metric are there dramatic shifts in algorithm corresponding to functional component shifts within the circuit. Moreover, all scores are relatively high, generally above 50%; the core solvers of the algorithm, copy suppression and name-mover heads, have scores above 70%. This suggests that analyses of circuits in fully pre-trained models may generalize well to other model states, rather than being contingent on the particular checkpoint selected.

[4]We omit Pythia-70m, as it does not learn the task; due to computational constraints, we omit Pythia-6.9b/12b.

Generalization across model scales also seems promising, as IOI circuit metrics from Pythia-160m are also high in larger Pythia variants. However, there is variation: while the name-mover, copy-suppression, and S-inhibition heads are at work in all models' circuits, the Pythia-160m circuit does not involve duplicate-token heads, while others do. So small differences exist amid big-picture similarity. Moreover, we stress that these algorithmic similarities might not hold for more complex tasks, for which a greater variety of algorithms could exist.

## 5 Graph-Level Circuit Analysis

We have now examined circuits over the training process from component and algorithmic perspectives. But how do the circuit subgraphs themselves change over time and scale? In Section 2.2, we explain how these subgraphs are collected; we applied this method to Pythia-70m through Pythia-2.8b for the tasks listed in Section 2.4. The result is a set of nodes and edges for each model, checkpoint, and task. Here, we briefly examine some trends we identified in the analysis of this data.

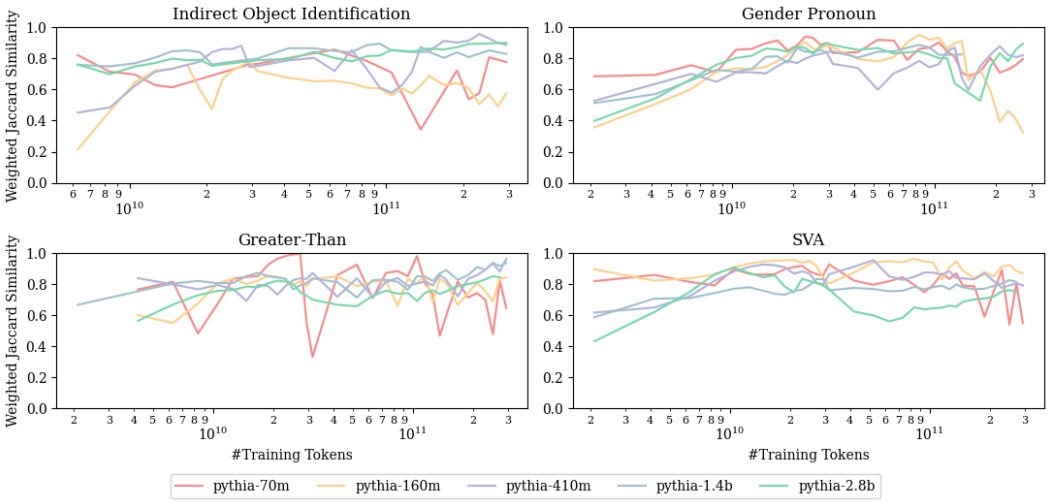


Figure 5: Exponentially-weighted moving average Jaccard similarity for circuit node sets over training token count. In general, larger models tend to have both higher average EWMA-JS and fewer abrupt fluctuations, indicating higher stability in the circuit constituents.

We first examine the consistency of the nodes in circuits over training. To measure this, we compute the Jaccard similarity (intersection over union) $x_t$ between the circuits at each given checkpoint and those at all previous checkpoints. In order to smooth out local fluctuations and observe longer-term trends, we apply an exponential weighting with a decay factor $\alpha = 0.5$, such that the value at a given checkpoint is the exponentially-weighted Jaccard similarity with the complete set of previous checkpoints. We calculate the weighted Jaccard similarity $\hat{x_t}$ at checkpoint $t$: $\hat{x_t} = 0.5\hat{x_{t-1}} + 0.5x_t$, Our results (Figure 5) suggest that larger models tend to form more stable circuits (with both higher average values and fewer sharp fluctuations); EWMA-Jaccard is more volatile for Pythia-70m/160m. In the Gendered-Pronoun circuit, we observe that significant changes can occur even late in training. We also computed how similar the circuit is during training with respect to the final circuit in Figure 7. we see more fluctuations, indicating that components swap during training at every checkpoint. Meanwhile, the upward trend indicates that circuits grow more and more similar to the final circuit during training.

We also compare the sizes (node counts) of the circuits over training. Across all four of our tasks, we find that the circuit size is positively correlated with the size of the models. We averaged node count across all the checkpoints for all the models on four tasks and calculated the pairwise correlation between the sizes of the models and the average node sizes. The Pearson correlation is $r$ =0.72 for IOI and SVA, 0.9 for Greater-Than, 0.6 for Gender Pronoun. We also find a high degree of variability—circuit sizes can remain stable or fluctuate significantly, with no clear pattern based on the model or the task. We leave further exploration of why this is the case to future work, but present our size metrics in Appendix C, Figure 6.